Slack Notification System: Complete Reference for BandChat

1. Core Philosophy

Slack's notification system is built on a single principle: **help users sort signal from noise**. The system manages user attention across multiple devices and contexts, ensuring important messages get through without overwhelming users with irrelevant pings.

Key stats that should inform BandChat's design:

- **Less than 5% of users ever customize notification settings.** Your defaults ARE the experience.
- Notification-related support tickets were historically Slack's worst-performing category for NPS scores and resolution time.
- The guiding question isn't "should we send a notification?" — it's "does the user want their attention drawn to the app right now?"

---

2. Notification Trigger Events

By default, Slack notifies a user when:

Event	Notification?
Someone sends them a **direct message**	Yes
Someone **@mentions** them by username	Yes
Someone uses **@channel** or **@here** in a channel they're in	Yes
Someone uses one of their **custom keywords**	Yes
Someone **replies to a thread** they're following	Yes
A new message is posted in a channel they're in (no mention)	No (bold in sidebar only)
A message is posted in a **muted** channel	No (unless @mention or keyword)

BandChat Mapping

For a band communication app, the equivalent triggers would be:

- **Direct message** → DM between band members

- **@mention** → Tagging a specific member (e.g., @Scott)
- **@channel/@here** → Notifying the whole band (e.g., @everyone, @band)
- **Keywords** → Custom alerts (e.g., "gig", "rehearsal", "setlist change")
- **Thread reply** → Reply in a conversation thread the user is participating in

---

3. Notification Delivery Channels

Slack delivers notifications through five distinct channels, with intelligent routing between them:

3.1 Desktop Banner Notifications

- Pop-up alerts on the computer screen, even when Slack isn't the active window
- Show who sent the message and in which conversation
- Disappear after a few seconds (configurable)

3.2 Sidebar / In-App Indicators

- **Bold text** on channel/DM names with unread activity
- **Red badge with number** for mentions, DMs, keywords, and thread replies
- **Red dot (no number)** for general unread activity
- Conversations with unread activity float to the top of the list

3.3 App Icon Badge

- Appears on desktop taskbar and dock icons
- **Dot** = unread activity in a workspace
- **Number** = direct mentions, DMs, keywords, or thread replies
- On Windows: appears in both taskbar and system tray

3.4 Mobile Push Notifications

- Delivered via APNs (iOS) and FCM (Android)
- Can be configured independently from desktop notifications
- Subject to a **timing delay** relative to desktop activity (see Section 5)

3.5 Email Notifications

- Enabled by default for new users who haven't installed the mobile app
- **Automatically disabled** once the user downloads the mobile app
- Covers mentions, DMs, and thread replies
- Acts as a fallback for users who are inactive for extended periods (2+ weeks)

BandChat Mapping

For a mobile-first band app, priority order should be:

1. **Push notifications** (primary — band members are mostly on phones)
2. **In-app badges and indicators** (bold names, unread counts)
3. **Email** (fallback for inactive members, event reminders)
4. Desktop banners are likely not needed for v1

---

4. The Notification Decision Flowchart

Slack's famous 2017 flowchart reveals the decision tree for every single message. Sophie Alpert simplified the original complex diagram into a cleaner linear flow. Here's the logic distilled:

Step 1: Is the channel muted?

- **Yes** → Skip to Step 2 (but still check for direct @mentions and keywords)
- **No** → Proceed

Step 2: Is the user in Do Not Disturb mode?

- **Yes** → Don't notify... UNLESS the sender explicitly overrides DND ("Notify Anyway")
- **No** → Proceed

Step 3: What is the user's notification preference for this channel?

Check in this order (first match wins):

1. **Channel-specific preference** (if set by user for this channel)
2. **Global preference** (user's default setting)
3. **System default** (mentions & DMs only)

The preference options are:

- **All new messages** → Every message triggers a notification
- **Mentions & DMs only** → Only @mentions, keywords, DMs, and thread replies
- **Nothing** → No notifications (effectively muted)

Step 4: Does the message qualify under the preference?

Check if the message matches:

- Is it a DM?
- Does it contain an @mention of this user?
- Does it contain @channel or @here (and are those not suppressed)?
- Does it contain one of the user's custom keywords?
- Is it a reply in a thread the user is following?
- Is it a highlight word match?

If **none match** under "Mentions & DMs" mode → Don't notify.

Step 5: Where should the notification be delivered?

- Is the user **active on desktop**? → Send desktop notification, suppress or delay mobile
- Is the user **inactive on desktop**? → Send mobile push notification
- Has the **mobile push timing threshold** passed? → Send mobile push
- Is the user **inactive everywhere** for 2+ weeks? → Send email

Step 6: Deliver the notification

- Format appropriately for the channel (banner, push, email)
- Include: sender name, channel/conversation name, message preview

Simplified Decision Pseudocode for BandChat

```
function shouldNotify(message, recipient):
    channel = message.channel

    // Step 1: Mute check
    if channel.isMutedBy(recipient):
        if not message.containsDirectMention(recipient)
           and not message.containsKeyword(recipient):
            return NO_NOTIFY
```

```
    // Step 2: DND check
    if recipient.isInDND():
        if not message.hasDNDOverride:
            return QUEUE_FOR_LATER  // or silent badge

    // Step 3-4: Preference check
    pref = recipient.getChannelPref(channel) ?? recipient.getGlobalPref()

    if pref == "everything":
        shouldAlert = true
    elif pref == "mentions_and_dms":
        shouldAlert = (
            message.isDM() or
            message.mentionsUser(recipient) or
            message.mentionsEveryone() or
            message.matchesKeywords(recipient) or
            message.isThreadReplyUserFollows(recipient)
        )
    elif pref == "nothing":
        shouldAlert = false

    if not shouldAlert:
        return UNREAD_BADGE_ONLY

    // Step 5: Delivery routing
    return routeNotification(recipient)

function routeNotification(recipient):
    if recipient.isActiveOnDesktop():
        return DESKTOP_BANNER
    elif recipient.mobileAppInstalled():
        if timeSinceDesktopActivity > MOBILE_DELAY_THRESHOLD:
            return MOBILE_PUSH
        else:
            return WAIT_THEN_MOBILE
    else:
        return EMAIL_FALLBACK
```

---

5. Cross-Device Intelligence

This is one of Slack's most sophisticated features and the hardest to replicate well.

Desktop-First Priority

When the user is active on desktop, Slack suppresses mobile push notifications entirely. This prevents the annoying "double ping" across devices.

Configurable Mobile Delay

Users can control when mobile notifications arrive relative to desktop inactivity:

- **Immediately** — push to mobile as soon as the message is sent
- **After inactive on desktop** (default) — wait until the user goes idle on desktop
- **After N minutes of inactivity** — configurable delay (e.g., 1, 2, 5, 10, 15 min)

Notification Rescission

If a push notification is sent to mobile but the user reads the message on desktop before opening it on mobile, Slack rescinds (removes) the mobile notification. This requires:

- Real-time read receipt tracking across devices
- Platform-specific push notification management (APNs silent pushes, FCM data messages)

BandChat Implications

For a mobile-first app, cross-device intelligence is less critical in v1. But you should still implement:

- **Read receipt syncing** — if a user reads a message in the app, clear the push notification
- **Badge count management** — decrement badge when messages are read
- **Silent push for badge updates** — update badge counts without alerting the user

---

6. User-Configurable Settings

6.1 Global Notification Preferences

- **What to notify about**: Everything / Mentions & DMs only
- **Desktop notifications**: On/Off
- **Mobile notifications**: On/Off (independent of desktop)
- **Notification sound**: Configurable per platform
- **Notification schedule**: Set specific days/hours for receiving notifications (e.g., weekdays 9am-6pm)

6.2 Per-Channel Overrides

Each channel can independently override global settings:

- **All new posts** — notify for every message in this channel
- **Just mentions** — only @mentions and keywords
- **Mute** — suppress all notifications (unread activity still tracked but no bold/badge)

Key UX detail: muted channels still show a badge if the user is directly @mentioned.

6.3 Custom Keywords

Users can set up keyword alerts — any message containing those keywords triggers a notification. Use cases in Slack: project names, client names, topics of interest. Keywords in thread replies don't trigger notifications.

6.4 Do Not Disturb / Pause Notifications

- Manual DND: click bell icon or use `/dnd` command
- Scheduled DND: automatic quiet hours (e.g., 10pm-8am)
- DND override: sender sees a prompt to "Notify Anyway" for urgent messages
- Teammates see a DND indicator, signaling not to expect an immediate response

6.5 Thread Following

Users automatically follow threads they've posted in or been mentioned in. They can manually follow/unfollow any thread. Thread replies to followed threads trigger notifications.

6.6 Notification Schedule

Set active hours for receiving notifications. Outside the schedule, notifications are paused (not lost — they queue up as unread).

BandChat MVP Settings

For v1, prioritize:

1. **Global toggle**: All messages vs. Mentions & DMs
2. **Per-band mute** (if multi-band support)
3. **DND / Quiet hours** (musicians have irregular schedules — this is critical)
4. **Thread following** (auto-follow threads you participate in)

Defer to v2:

- Custom keywords
- Per-channel overrides (unless BandChat has sub-channels)
- Cross-device routing logic

---

7. Visual Indicator System (In-App)

Badge Types

Indicator	Meaning	Visual
Bold channel name	Unread messages exist	Text weight change
Red dot	Unread activity (general)	Small circle on icon
Red number badge	Unread mentions/DMs/keywords	Number in red circle
Blue dot	Unread messages in channel (Slack mobile)	Subtle indicator

Sidebar Behavior

- Channels with unread activity sort to the top (mobile)
- Bold text for any unread messages
- Badge count for actionable items (mentions, DMs)
- Muted channels: no bold, no badge (except direct @mentions)

BandChat Indicator Design

For a band messaging app:

- **Bold conversation name** + preview text for unread messages
- **Numeric badge** on the app icon for unread DMs and @mentions
- **Dot indicator** on specific conversations with unread activity
- Consider color coding: red for urgent (@everyone), blue for regular unread

---

8. Notification Content & Formatting

Push Notification Content

Slack push notifications include:

- **Title**: Channel name or sender name (for DMs)
- **Body**: Sender name + message preview (truncated)
- **Action buttons**: Reply, Mark as Read (on supported platforms)
- **Deep link**: Tapping opens the specific conversation/message

Notification Grouping

- Multiple messages from the same channel are grouped
- iOS: uses thread identifiers for grouping
- Android: uses notification channels for categorization

BandChat Notification Content

```
Title: "Frozen Assets"                    // Band/group name
Body:  "Taka: Hey, rehearsal moved to 7pm Thursday"  // Sender + preview
```

For @mentions:

```
Title: "Frozen Assets"
Body:  "Taka mentioned you: @Scott can you bring the amp?"
```

---

9. Infrastructure Architecture (High Level)

Slack's notification pipeline flows through these systems:

1. **Message arrives** → Webapp (application logic)
2. **Notification decision engine** → Evaluates the flowchart logic
3. **Job queue** → Queues notifications for async processing
4. **Push service** → Routes to appropriate delivery channel
5. **Third-party services** → APNs (iOS), FCM (Android), email providers
6. **Client receives** → iOS, Android, Desktop, Web

Key Engineering Considerations

- Slack traces every notification flow at 100% sampling (not sampled like regular requests) for debugging
- The notification flow executes across multiple request contexts — it doesn't map cleanly to a single request
- For @here/@channel in large channels, a single message can generate hundreds of thousands of individual notification decisions
- Notification-related bugs are among the hardest to debug because they span so many systems

BandChat Architecture (Recommended)

Since BandChat is Firebase-based:

1. **Message written to Firestore** → triggers Cloud Function
2. **Cloud Function runs notification logic** → evaluates user preferences, DND, mute status
3. **FCM (Firebase Cloud Messaging)** → sends push to iOS/Android
4. **Client handles** → displays notification, updates badge count
5. **Read receipts** → client sends read confirmation, triggers badge/notification cleanup

---

10. Context-Aware Controls (Advanced UX)

One of Slack's smartest patterns is presenting notification settings **at the moment they become relevant**, rather than burying them in a settings screen.

Examples:

- **First weekend notification** → Slack asks: "Want to keep getting notifications on weekends?"
- **Noisy channel** → Slack surfaces a mute option directly in the notification area
- **Frequent @mentions** → Options to adjust mention sensitivity appear in context

BandChat Opportunities:

- **First notification from a new band** → "Want to get notified for all messages or just when you're mentioned?"
- **After a burst of messages** (e.g., 20+ in an hour) → "This chat is active. Mute for now?"

- **After hours** → "You're getting pinged late. Set quiet hours?"

---

11. Anti-Patterns to Avoid

Based on Slack's learnings:

1. **Don't growth-hack via notifications.** More notifications ≠ more engagement. If users disable notifications at the OS level, you lose the channel permanently.
2. **Don't duplicate across devices.** If the user read it on one device, clear it everywhere.
3. **Don't notify for your own messages.** This seems obvious but is a common bug.
4. **Don't batch-delay urgent notifications.** DMs and @mentions should arrive promptly; batching is only for low-priority activity.
5. **Don't send notifications for edited messages** (unless the edit adds a new @mention).
6. **Don't over-notify for reactions/emoji.** Slack does NOT push-notify for reactions. Keep it as in-app only.

---

12. BandChat v1 Notification Spec (Recommended)

Based on all of the above, here's a recommended notification spec for BandChat v1:

Trigger Events

- New DM → Push notification
- @mention in group → Push notification
- @everyone in group → Push notification
- New message in group (no mention) → Badge/bold only, no push
- Thread reply (if participating) → Push notification

User Settings (Settings Screen)

- **Notify me about**: All messages / Mentions & DMs only (per band)
- **Quiet hours**: Enable + start/end time
- **Mute band**: Per-band mute toggle

Push Notification Format

- Title: Band name (or sender name for DMs)

- Body: "Sender: message preview..."
- Tap action: Deep link to conversation
- Badge: Increment on receive, decrement on read

In-App Indicators

- Bold conversation name for unread
- Numeric badge for mentions/DMs
- "New messages" divider line in chat scroll

Infrastructure

- Firestore trigger → Cloud Function → FCM
- User preference document in Firestore (per-user, per-band)
- Read receipt writes clear badge via silent push

Reference sources: Slack Help Center, Slack Engineering Blog (Tracing Notifications), Courier.com interview with Liza Gurtin (former Slack notifications product lead), Sophie Alpert's simplified flowchart analysis.